

Business Requirement Coverage

The 73 user stories judged against the business problem they exist to solve — not against the code. Does this backlog, if fully delivered, actually give the bank what it asked for?

REVIEWER	DATE	SOURCE OF TRUTH	DAYS TO SUBMIT
Shanthi Subramanian · BA / QA	23 September 2026	The client brief	7

10/10 FUNCTIONAL REQS COVERED	5/5 CONSTRAINTS COVERED	2/4 BUSINESS OUTCOMES SERVED	7 BUSINESS GAPS
---	---	--	---------------------------------------

What this reviews, and against what

A different question from the last review — and a harder one.

The previous review asked *"is each story built and tested?"* This one asks *"if every story on this board were delivered perfectly, would the bank's problem be solved?"* A backlog can be fully implemented, fully tested, and still miss the point.

The source of truth

The business requirements are the client brief in `CLAUDE.md` — the problem statement as given. It states four things a BA can hold the backlog against: **the business case** (why this exists), **the functional requirements** (what the platform must do), **the non-negotiable constraints**, and **the actors**.

ONE THING TO KEEP IN MIND THROUGHOUT

The brief says the problem statement is **"deliberately incomplete — expansion is graded."** So a gap between the brief and the backlog is not automatically a failure. The question for each gap is: *was it filled deliberately, filled accidentally, or left open without anyone deciding?* The third is the only one that is actually a problem — and this review separates them.

How each requirement was rated

RATING	MEANS
SERVED	One or more stories exist, and together their criteria would satisfy the requirement
PARTIAL	Stories exist but leave a material part of the requirement unaddressed
NOT SERVED	No story on the board addresses it

Whether a story is *built* is irrelevant here. This is about whether the backlog, as written, is aimed at the right target.

The business case

Four stated problems. The backlog solves two of them well, one implicitly, and does not address one at all.

Runs on email/Word/Excel/SharePoint. Assessment takes 15–20 business days. Two analysts can reach different conclusions on the same change. Reconstructing the reasoning behind a past rating (for examiners) is slow and incomplete.

BC	THE STATED PAIN	COVERED BY	RATING
BC-1	Fragmented tooling — email, Word, Excel, SharePoint	Epics 1–10 deliver one governed platform end to end. Epic 14 replaces the "go ask another system" step	SERVED
BC-2	15–20 business day cycle	Nothing. No story sets a target, measures elapsed time against one, escalates a breach, or reports on throughput	NOT SERVED
BC-3	Two analysts reach different conclusions	Helped implicitly by AI category proposal and a deterministic scoring formula — but no story names consistency as a goal or measures it	PARTIAL
BC-4	Examiner reconstruction is slow and incomplete	US-9.1 (full history), US-9.2 (immutability), US-9.3 (examiner-ready export)	SERVED

BC-2 IS THE HEADLINE BUSINESS BENEFIT, AND NOTHING MEASURES IT

"15–20 business days" is the number a sponsor would quote to justify funding this platform. The backlog contains **no target, no service level, no escalation, and no reporting** against it. I confirmed this in the data model: there is no `due_date` , `sla` , `target_date` or `cycle_time` anywhere in the schema.

The system does show *days elapsed* on a Product Owner's request list — but that is a number on a screen with nothing to compare it to. At the end of this project nobody will be able to say whether the platform made assessments faster, because nothing records what "faster" would mean.

BC-3 IS SERVED BY ACCIDENT RATHER THAN BY DESIGN

Two mechanisms genuinely reduce inconsistency: every assessment starts from the *same* AI-proposed categories against the *same* citable framework, and the score is a fixed formula rather than a judgement. Both are real, and both are good design.

But no story says "analysts should reach consistent conclusions", so nothing verifies it and nothing protects it. If someone later made scoring configurable per analyst, no acceptance criterion anywhere would object.

Consistency is currently an emergent property, not a requirement — and emergent properties are the ones that quietly disappear.

Functional requirements

The brief lists ten things the platform must do. All ten have an epic. This is the backlog's strongest area.

FR	REQUIRED BY THE BRIEF	EPIC	RATING	NOTE
FR-1	Intake a change request with defined types	Epic 1	SERVED	All six types enumerated and enforced in the data model
FR-2	Decompose into categories from a supervisory framework	Epic 2	SERVED	FFIEC, with citations per category
FR-3	Retrieve / surface relevant policy	Epic 3	SERVED	Deliberately deterministic search rather than AI — a defensible judgement call
FR-4	Extract structure from submitted documents	Epic 4	SERVED	Source excerpt + confidence per field
FR-5	Draft the assessment for analyst review	Epic 5	SERVED	Per category, not one monolithic narrative
FR-6	Analyst edit / override with reason, first-class	Epic 6	SERVED	Three stories; the rule also recurs across Epics 2, 4, 7, 10
FR-7	Score: controls mitigate, never eliminate	Epic 7	SERVED	Enforced at calculation <i>and</i> configuration, as the brief demands
FR-8	Route to committee with a recorded vote	Epic 8	SERVED	All four outcomes the brief names are implemented as allowed values
FR-9	Full immutable audit trail	Epic 9	SERVED	Append-only at the data layer, as the brief specifies
FR-10	Tunable scoring / workflow configuration	Epic 10	PARTIAL	Scoring is tunable; escalation thresholds are named in US-10.2 but nothing implements them

WORTH SAYING PLAINLY

Ten out of ten functional requirements have a home, and the mapping is one-to-one with the brief's own list. The four decision outcomes the brief names — approved, rejected, deferred, approved-with-conditions — appear verbatim as database constraints. That is textbook traceability from a client brief to a working system, and it is the single strongest thing about this backlog.

The AI requirement, specifically

The brief is precise about where AI belongs: *"mapping the request to risk categories, finding relevant policy, extracting structure from submitted documents, and drafting the assessment narrative."* Four named touchpoints.

TOUCHPOINT	EPIC	IMPLEMENTED AS
Map to risk categories	2	AI grounded against a fixed category list
Find relevant policy	3	DETERMINISTIC SEARCH — a deliberate deviation
Extract from documents	4	AI with confidence and source excerpt
Draft the narrative	5	AI with a fabrication guard

Epic 3 is the interesting one: the brief asked for AI, the team chose full-text search instead, and *documented why*— ranking indexed text is a solved problem, and an LLM would add latency, cost and nondeterminism for no accuracy gain. **That is a deviation from the brief, and a good one** — but it is a decision the client should be told about rather than discover.

Non-negotiable constraints

Five rules the brief states as absolute. All five are reflected in the backlog.

NC	THE RULE	WHERE IT LIVES IN THE BACKLOG	RATING
NC-1	The system prepares, humans decide. Nothing auto-approved or auto-rejected	Every AI epic (2, 4, 5) pairs with a human gate; US-6.3 is the single chokepoint; US-8.2 requires a human vote	SERVED
NC-2	Risk decomposition grounded in a real, published, citable framework	US-2.1 requires a citation per category; FFIEC seeded with section references	SERVED
NC-3	Residual risk never zero , at calculation and configuration time	US-7.1 (calculation), US-10.1 (configuration) — the brief demands both and both exist	SERVED
NC-4	Override any AI output , state a reason, retain both	US-2.2, US-4.2, US-6.1, US-7.2, US-10.1	SERVED
NC-5	Synthetic data only. No connection to any real system	Epic 14 — mock CRM, Core Banking and Vendor Management as a separate service	SERVED

A BA OBSERVATION ON NC-4

The override rule is the most *widely spread* requirement in the brief — it appears in five separate stories across four epics, and the brief calls it first-class. But it exists only as a repeated acceptance criterion; **no single story owns it as a cross-cutting rule.**

That is why it ended up the least-verified rule in the product (from the previous review, only one of the five has a test that a blank reason is refused). A requirement spread across five stories and owned by none is the classic way a non-negotiable quietly becomes optional.

Business gaps

Seven things the business needs that no story currently delivers. Ordered by business consequence, not effort.

BG-1 **HIGH** BUSINESS CASE

Nothing measures the cycle time the project exists to reduce

No target, no service level, no escalation on a breach, no throughput reporting. Confirmed absent from the data model — no `due_date`, `sla`, `target_date` or `cycle_time` column exists.

The consequence is not technical. At the end of this project, **no one can demonstrate the platform made anything faster** — which is the benefit the business asked for.

NEEDS

One story: a target cycle time per change type, elapsed time measured against it, and a simple view of requests approaching or breaching it

EFFORT

Low — `submitted_at` already exists; this is arithmetic and a screen

BG-2 **HIGH** DELIVERY

The evaluation framework has no user story at all

The brief makes AI output quality a first-class deliverable, and the project's own success criteria weight it at **10%** — equal to requirement expansion and double production readiness. An evaluation harness exists in the codebase with 12 test cases across the three AI touchpoints.

But **no epic and no story on the board covers it**. It is real work, it is graded, and it is invisible to anyone reading the backlog — which means nobody is accountable for finishing it, and it does not appear in any plan.

NEEDS

An epic with stories for: evaluating each AI touchpoint against fixed datasets, running the evaluation in the pipeline, and acting on a regression

EFFORT

Low to write — the work largely exists and simply is not represented

BG-3 **MEDIUM** BUSINESS CASE

Analyst consistency is an outcome, not a requirement

"Two analysts can reach different conclusions on the same change" is one of three stated pains. The design genuinely helps — a shared framework, a shared AI starting point, a deterministic formula — but no story names consistency, so nothing protects it from a future change.

NEEDS

Either a story that states it as a requirement, or an explicit note that it is served by design decisions in Epics 2 and 7 — so it is a recorded decision rather than luck

BG-4 MEDIUM FUNCTIONAL

Notifications are required by three stories and built by none

US-1.3 requires the requester be notified on status change; US-8.1 requires notification when an assessment is routed; US-8.3 requires the Product Owner be told the decision and any conditions. There is no notification mechanism anywhere in the system.

This matters more than it looks: the brief's first pain is that the process runs on *email*. If the platform cannot tell anyone anything, users will go back to email to find out what happened — and the original problem returns through the back door.

NEEDS

A decision first: in scope or not? If not, remove the criteria from three stories with a stated reason. If yes, one story owns it

BG-5 MEDIUM FUNCTIONAL

Escalation thresholds are specified but not implemented

US-10.2 names "escalation thresholds" among the workflow rules an analyst should be able to tune. The schema mentions `EscalationThreshold.ResidualRating` — *as a comment*. No such rule is seeded and nothing consumes one. The only workflow rule that exists is the committee quorum.

Business meaning: a very high residual risk currently follows exactly the same path as a trivial one. There is no "this one needs senior attention" mechanism, which is a normal expectation in a risk function.

BG-6 MEDIUM ORGANISATIONAL

The bank has four business units; the system has none

The brief describes the client as running *consumer banking, commercial banking, payments and wealth management*, and says a business unit raises the change. The application data model has **no concept of a business unit** — a change request records who submitted it, but not which part of the bank it came from.

Consequences: no routing by business unit, no BU-specific workflow rules, and no way to answer "how much risk is the payments business generating?" — a question any FCRM function would ask. It also touches open question 2, which asked whether the framework varies by business line.

NOTE

Defensible to descope for a hackathon — but it should be a stated decision, since the brief introduces the business units explicitly

"FCRM Leadership" appears as an actor with no role in the system

The brief names three user roles: Product Owner, FCRM Analyst, Risk Committee. The system implements four (adding Admin). Several stories — including US-9.2 and three raised by QA — are written from the perspective of *FCRM Leadership*, who has no role, no screen and no permission set.

Minor, but it means those stories have no one who can actually perform them. Either leadership is the Admin role, or the persona should be changed to one that exists.

Scope balance

Where the backlog's weight actually sits — and whether that is right.

GROUP	STORIES	SHARE	SERVES
The business problem — Epics 1–10, 14	30	41%	What the bank asked for
How it gets delivered — Epics 15–18	36	49%	Infrastructure, pipelines, observability
Quality & governance — Epics 11–13	6	8%	Access control, deployment, non-functional
Documentation	1	2%	—

Nearly half the backlog is about delivery rather than the business problem. On a normal bank engagement that ratio would be a serious warning sign — the client is paying for risk assessment, not for Terraform modules.

HERE IT IS CORRECT, AND WORTH BEING ABLE TO EXPLAIN

This project's success criteria weight **SDLC automation across all six stages at 20%** and production readiness at 5% — a quarter of the assessment is about *how* it was built. The infrastructure epics are not scope creep; they are a graded deliverable, and documenting them as stories is what makes that work visible.

The point worth carrying into any review conversation: this ratio is **deliberate and defensible here**, and would not be on a real engagement. Saying so demonstrates the judgement, rather than leaving a reviewer to wonder whether the team lost sight of the customer.

Are any stories not traceable to a requirement?

No. Every one of the 73 traces either to a functional requirement in the brief, a non-negotiable constraint, or the project's own delivery criteria. **There is no scope creep** — the backlog has gaps, not excess.

Verdict & actions

Would this backlog, fully delivered, solve the bank's problem?

VERDICT

Substantially yes — with one significant caveat. All ten functional requirements and all five non-negotiable constraints are covered, traceably, with the brief's own vocabulary carried into the data model. A reviewer comparing the brief to the board would find the mapping clean.

The caveat is that the backlog delivers *the platform* the brief describes but not *the improvement* the brief justifies. The bank's stated pain is a 15–20 day cycle and inconsistent outcomes. The backlog would replace the tooling, and almost certainly help with both — but it contains nothing that would let anyone **demonstrate** either.

Recommended actions

#	ACTION	CLOSES	EFFORT
1	Add a story for cycle time against a target per change type	BG-1 · the headline business benefit	Low
2	Add an epic for the AI evaluation framework	BG-2 · 10% of the assessment, currently invisible	Low — the work exists
3	Decide on notifications — build one story or remove three sets of criteria	BG-4 · three stories currently cannot be completed	Decision first
4	Record analyst consistency as a served requirement, naming the design decisions that serve it	BG-3	Very low
5	Record the Epic 3 deviation — the brief asked for AI, the team chose deterministic search	Turns a deviation into a defensible decision	Very low
6	State explicitly that business unit and escalation thresholds are descoped , or schedule them	BG-5, BG-6	Decision only
7	Resolve the FCRM Leadership persona to a role that exists	BG-7	Very low

Six of the seven are writing or deciding rather than building. With seven days left, that matters: **BG-1 and BG-2 are the two worth actually building**, and both are small.

STILL OPEN FROM EARLIER REVIEWS

Three questions remain unanswered and each blocks acceptance criteria: **Q3** (does a scoring override need secondary sign-off), **Q5** (who approves configuration changes), and **Q6** (who owns database schema deployment). Q6 is the urgent one — it is an unassigned responsibility, not an open design question, and it currently blocks any deployment at all.

Reviewed 23 September 2026 against the client brief in `CLAUDE.md`, the 91-item Azure DevOps board, and the application data model. Requirements were derived from the brief's own structure — business case, functional requirements, non-negotiable constraints, actors — and each was rated on whether the backlog *as written* would satisfy it, independently of whether any story is built. Absence of concepts such as SLA and business unit was confirmed against the database schema rather than inferred. Companion document: *Story-by-Story Board Review*, which covers build and test status.